

Efficient Learning of Relational Gaifman Models using Probabilistic Logic

Devendra Singh Dhami, Gautam Kunapuli, and Sriraam Natarajan

The University of Texas at Dallas

`{devendra.dhami,gautam.kunapuli,sriraam.natarajan}@utdallas.edu`

Abstract. We consider the problem of structure learning for Gaifman models and learn relational features using probabilistic logic methods that can then be used to derive feature representations from a knowledge base. These relational features are first-order rules that are then partially grounded and counted over local neighborhoods of a Gaifman model to obtain the feature representations. We propose a method for learning these first-order features for a Gaifman model by using first-order tree distances. Our empirical evaluation on real data sets demonstrates the superiority of our approach over classical rule-learning.

Keywords: first-order tree distance · relational Gaifman models · relational learning.

1 Introduction

Learning embeddings of large knowledge bases has become a necessity due to the importance of reasoning about objects, their attributes and relations in large graphs. Naturally, Statistical Relational AI [34,12] that has the ability to reason about multi-relational data in the presence of uncertainty, has been successfully applied to knowledge extraction tasks from relational data. While specific models such as Markov Logic [35], ProbLog [8] and PSL [4] (to name a few) exist, a more general, scalable model based on Gaifman’s locality theorem [32] was proposed recently and is capable of handling complex multi-relational data.

We consider learning in this framework. While the previous work used a handcrafted relational graph structure and learned the parameters, in this work, we explore the full model learning of these models. Specifically, we consider three different approaches. (1) As suggested by Niepert [32], we employ Inductive Logic Programming (ILP) [27] to learn discriminative first-order rules; (2) Inspired by the success of random walks in deep relational models [21,22,16], we employ first-order random walks; we employ an in-house strategy of constructing first-order clauses using a random walk strategy. Unlike previous work, by virtue of employing a logic learner, we can clauses with n-ary predicates. (3) Finally, as a novel contribution, we propose the use of paths from first-order trees (similar to Tilde trees [2] learned using one-class classification for logic models [18]; specifically, each path from root to leaf of a tree can be considered as a first-order feature. Given these logical features, we construct propositional features

by counting the satisfied groundings of the first-order rules and then apply traditional discriminative learning algorithms such as Gradient Boosting and Logistic Regression.

2 Background and Related Work

A **grounding** of a predicate with logical variables $x_1, \dots, x_k$ is a substitution $\{\langle x_1, \dots, x_k \rangle / \langle X_1, \dots, X_k \rangle\}$ mapping each of its variables to a constant in the population of that variable. A **knowledge base** $\mathcal{B}$ consists of (1) a finite domain of objects $\mathcal{D}$ (also known as entities), (2) a set of predicates $\mathcal{R}$ that describe the attributes and relationships of the objects, and (3) an interpretation that assigns a truth value to every grounded predicate.

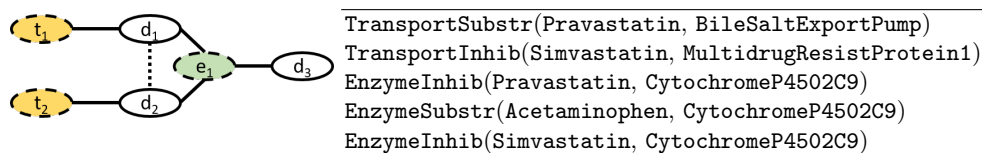


Fig. 1. An example Gaifman graph for a drug-drug interaction (DDI) knowledge base. Here $d_1, d_2, d_3 = \{\text{Pravastatin, Simvastatin, Acetaminophen}\}$, $t_1 = \{\text{Bile salt export pump}\}$, $t_2 = \{\text{Multidrug resistance protein 1}\}$ and $e_1 = \{\text{Cytochrome P450 2C9}\}$. Note that the dotted line between d_1 and d_2 is the link we want to predict.

Discriminative Gaifman Models: The Gaifman graph $\mathcal{G}$, also known as the primal graph, of a knowledge base $\mathcal{B}$ is an **undirected graph**, where the nodes are the entities $e \in \mathcal{D}$. $\mathcal{G}$ contains edges joining two nodes only if the entities a and b corresponding to those nodes are present in a relation together $R(\dots, a, \dots, b, \dots) \in \mathcal{B}$. $\mathcal{G}$ can be used to easily identify co-occurrences (or lack thereof) among every pair of entities in $\mathcal{B}$.

Furthermore, cliques in $\mathcal{G}$ group entities that co-occur pairwise through shared relationships, and such cliques capture the local structure of a knowledge base. We illustrate this in Figure 1, which shows a knowledge-base fragment and the corresponding Gaifman graph for drug-drug interaction (DDI). Given entities (drugs, enzymes, transporters) and relations between them, the underlying machine-learning task is to predict if two drugs interact. The dotted line represents the target predicate, and identifying it corresponds to **link prediction**.

The distance $d(a, b)$ between two nodes $(a, b) \in \mathcal{G}$ is the minimum number of hops required to reach node b from node a . For example, in Fig. 1, $d(d_1, t_1) = 1$ and $d(d_1, d_2) = 2$. The r -neighborhood of a $a \in \mathcal{G}$ is the set of all nodes that are at most a distance r from a in the Gaifman graph: $N_r^{\mathcal{G}}(a) = \{\bar{a} \in \mathcal{G} \mid d(a, \bar{a}) \leq r\}$. For example, $N_1(d_1) = \{t_1, e_1\}$ and $N_2(d_1) = \{t_1, e_1, d_2, d_3\}$. Figure 2 shows the 1 and 2-neighborhood of a node (colored red) in a given Gaifman model.

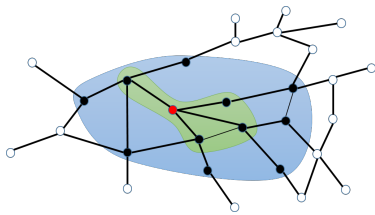


Fig. 2. Gaifman neighborhoods.

When a first-order rule/clause $\varphi(x)$ is relativized by the neighborhood of the free variable x , the resulting first-order rule $\psi^{N_r(x)}(x)$ is called r -local. A Gaifman neighborhood can be thought of as representing second-order proximity between nodes. The interpretation is that nodes with shared neighbors are more likely to be similar and more likely to have a link between them. Discriminative Gaifman Models (DGMs, [32]) are relational models that can exploit structural features of a local neighborhood of a knowledge base. These structural features

are aggregated from locally-sampled neighborhoods, and the aggregation is based on the *Gaifman locality theorem* [11] stated as: *Every first-order sentence is logically equivalent to a Boolean combination of basic r -local sentences.* An r -local sentence is of the form $\exists x_1 \dots \exists x_k \left(\bigwedge_{1 \leq i < j \leq k} d(x_i, x_j) > 2r \wedge \bigwedge_{1 \leq i \leq j} \varphi(x_i) \right)$, where $r, j \geq 1$ and φ is an r -local first order formula. In simpler terms, the locality theorem states that only a small part of a given structure is relevant for evaluating a query statement and thus a global structure search is not required. For example, if querying about the drug d_1 in Figure 1, a search within the 1-neighborhood of e_1 (say), that is $\{t_1, e_1\}$ is more relevant than searching through the complete graph which can be greatly computationally inefficient. Another way to look at the theorem is: *a first-order rule is true if it is true in the local r -neighborhoods of a given graph.* The DGM approach uses the Gaifman locality theorem to generate low-level embeddings for a given knowledge graph, which can then be used as propositional features in a standard classifier. Learning embeddings from a given knowledge graph, or **graph embedding**, is a well-studied problem in machine learning. A large body of recent work in this area can be categorized broadly by the underlying approaches: matrix factorization, deep learning, edge reconstruction, graph kernels and generative models. These approaches have been extensively surveyed recently; see for instance, Nickel et al. [31], Wang et al. [38] and Cai et al. [5]. In general, Gaifman models tend to scale better than many such approaches to higher-arity relations and target-query complexity [32] owing to their local view and incorporation of count-based features.

Relational and Structure Learning: Relational learning is the task of predicting values of a relation, given a relational database of entities and the observed relations. One of the most important tasks in relational learning is that of *link prediction* which determines whether a relation (link) exists between entities based on the given relational database. Taskar et al. [37] use template graphical models to predict links between entities in relational domains with a focus on collective link classification. Martinez et al. [25] and Hasan et al. [1] present a comprehensive survey on link prediction problems in complex networks and social networks respectively. Deep learning has also been used for link prediction, especially via graph neural networks [14] whereas Chuan et al. [6] present a

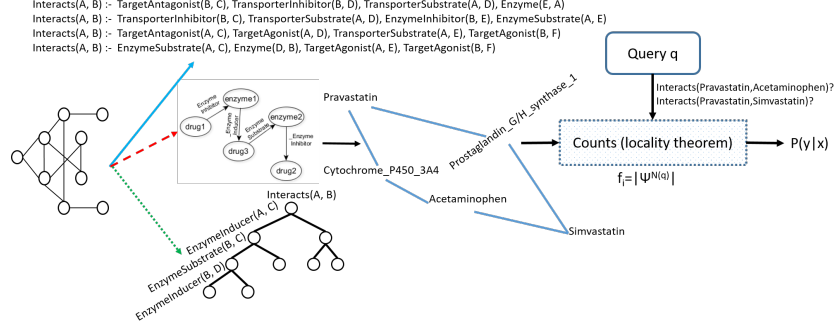


Fig. 3. A general overview of our overall approach using Gaifman models.

metric-learning method for link prediction in online social networks. Other recent methods on link prediction are surveyed in [13].

Structure learning is the problem of identifying a graph structure to principally summarize dependencies in a data set. Structure learning (in relational/probabilistic graphical models) can be interpreted as learning first-order rules from the data. Structure learning for Bayesian networks has been especially well studied and various methods exist such as genetic algorithms [23], linear programming [15] and constraints [7]. For bi/undirected graphical models, local learning can be facilitated using gradient-boosting [29,17]. Several other methods also exist for learning undirected (relational) models [24] [19]. A more comprehensive survey for structure learning in graphical models is provided in [39].

3 Structure Learning for Discriminative Gaifman models

Given: A knowledge base $\mathcal{B}$, facts F_s , and corresponding Gaifman graph $\mathcal{G}$;
Output: A DGM $\mathcal{M}$ that is trained for a particular link prediction task $\mathcal{T}$;
To Do: Construct a set of first-order features Φ , and train a discriminative learner to predict $\mathcal{T}$.

Our approach (1) constructs first-order rules Φ that form the base set of features; (2) instantiates the rules and performs counting based on task $\mathcal{T}$ to construct propositional features $\mathcal{F}$; and finally, (3) learns a discriminative classifier with $\mathcal{F}$ (Figure 3).

3.1 Learning First-Order Rules

Given a knowledge base $\mathcal{B}$, the Gaifman graph $\mathcal{G}$ is obtained by instantiating the entities that are connected by an edge type (relation) together in the form $R(e_1, e_2)$, that is, $relation(type_1, type_2)$. The relation (link) to be predicted, defined by the target predicate, forms the set of positive examples. We make the *closed-world assumption*, that is, unobserved edges in the graphs are negative examples. Each relational example also has *facts* associated with it, which are the ground predicates in $\mathcal{B}$ that describe relational example, its attributes and relationships. All such facts are denoted F_s .

Features via Relational Rule Learning. Our first solution is inspired by Niepert [32], who suggested the use of an Inductive Logic Programming (ILP)

style learning method. This method learns a set of discriminative Horn clauses (implications of the form *if* $\langle condition \rangle$ *then* $\langle consequence \rangle$). Specifically, we use an ILP system called WILL [30] to learn the first-order features¹. WILL first selects an example from the set of all examples and then finds a clause (rule) that *best covers* the examples. The *best covering* is the most general clause that maximizes the difference between the number of positive and negative examples covered². Each *best covering* clause becomes a first-order rule in our model. The examples covered by the clause are then removed and the process is repeated till a stopping criterion is satisfied; for example, we have extracted a maximum number of rules/clauses. Note that when a stopping criterion requires n rules to be extracted, it is sometimes possible to extract $m < n$ rules that cover the examples adequately.

Features via Relational Random Walks. Relational data is often represented using a graph that defines a domain’s schema; in such a representation, a relation $R(e_1, e_2)$ is a predicate edge between two entity type nodes: $e_1 \xrightarrow{R} e_2$. A relational random walk (RW) through a graph is a chain of such edges corresponding to a conjunction of predicates. For a random walk to be semantically sound, we should ensure that the input type (domain) of the $i + 1$ -th predicate is the same as the output type (range) of the i -th predicate. An example relational random walk from the drug-discovery domain is:

$$\text{Interacts}(d0, d3) \Leftarrow \text{TargetInhib}(d0, t0) \wedge _ \text{TargetInhib}(t0, d1) \wedge \\ \text{TransporterSubstr}(d1, t2) \wedge _ \text{TransporterInhib}(t2, d3).$$

This is a semantically sound random walk as it is possible to chain the second argument of each predicate to the first argument of the succeeding predicate. This random walk also contains *inverse predicates* (prefixed by an underscore, such as $_ \text{TargetInhib}$). Inverse predicates are distinct from their corresponding predicates as their arguments are *reversed*. Thus, this relational random walk chains the first variable $d0$ in the target predicate $\text{Interacts}(d0, d3)$ with the second variable $d3$. The chain represents a relational feature and constitutes a random local structure of the form:

$$d0 \xrightarrow{\text{TargetInhib}} t0 \xrightarrow{_ \text{TargetInhib}} \dots d1 \xrightarrow{\text{TransporterSubstr}} t2 \xrightarrow{_ \text{TransporterInhib}} d3.$$

Thus, to construct a first-order random walk, only the schema describing $\mathcal{B}$ is required. We adapt path-constrained random walks (PCRW, [21]) to construct these random walks. The algorithm starts at the first entity in the target relation, and makes a walk over the (parameterized) graph to end at the second entity present in the target relation. One limitation of PCRW is that the random walks are only performed over binary relations. However, since we employ a predicate representation, *we generalize and learn with arbitrary n -ary relations*.

Features via First-Order Tree Distance (FOTD). A common issue in many real-world relational domains, especially knowledge bases, is that only

¹ Any other ILP learner such as [36], [33] or [28] could be used.

² Ideal coverage means *all* positive examples and *no* negative examples which can easily overfit.

“positive” instances of a relation are annotated, while “negative” instances are not explicitly identified. This is because the number of instances where the relation does not hold is very large, and annotation can be prohibitively expensive. Learning with highly imbalanced data sets requires reasoning over just the positive instances. Intuitively, if we can construct a first-order classifier describing the positive examples, then rules characterizing this classifier are essentially features that describe positive examples. Such classification typically requires a *distance measure* to characterize the density of the positive class. While, for standard vector and matrix data, many different distance measures exist, the issue is far more challenging for relational data, and depends on the underlying representation of the classifier.

Suppose we use an off-the-shelf learner to learn first-order trees [3] to describe each class in the data. Such relational trees form a decision-list of first-order rules. These trees can then be used to compute the *first-order distance* between a pair of examples x_1 and x_2 ,

$$d(x_1, x_2) = \begin{cases} 0, & \text{if LCA}(x_1, x_2) \text{ is a leaf;} \\ e^{-\lambda \cdot \text{depth}(\text{LCA}(x_1, x_2))}, & \text{otherwise,} \end{cases} \quad (1)$$

where LCA refers to the *least common ancestor* of the examples x_1 and x_2 . Figure 4(a) shows examples $x_1 \equiv \text{advisedBy}(\text{Tom}, \text{Mary})$ and $x_2 \equiv \text{AdvisedBy}(\text{Tom}, \text{John})$; they both follow the same path down the tree before diverging at a node at depth 2. Now, consider x_1 and $x_3 \equiv \text{AdvisedBy}(\text{Ada}, \text{Dan})$. In this case, we have that the least common ancestor is at depth 1. Since the distance measure is inversely related to depth of the least common ancestor, we have that x_1 and x_2 are closer together than x_1 and x_3 . Typically, more than one tree is learned (say, via functional gradient boosting), and the first-order classifier is a weighted combination of these trees. Then, the overall distance function is simply the weighted combination of the individual tree-level distances: $D(x_1, x_2) = \sum_i \beta_i d_i(x_1, x_2)$ where β_i is the weight of the i^{th} tree and $\sum_i \beta_i = 1, \beta_i \geq 0$. The non-parametric function $D(\cdot, \cdot)$ is a first-order distance measure learned on the data. The distance function can then be used to compute the density estimate for a new example z as a weighted combination of the distance of z from all training examples x_j , $E(z \notin \text{class}) = \sum_j \alpha_j D(x_j, z)$, where α_j is the weight of the labeled example x_j and $\sum \alpha_j = 1, \alpha_j \geq 0$. Note that expectation above is for $z \notin \text{class}$, since the likelihood of class membership of z is inversely proportional to its distance from the training examples describing that **class**.

We learn a tree-based distance iteratively [18] to introduce new first-order features. The left-most path in each tree is a conjunction of predicates, that is, a clause, which can be used as a first-order feature. Figure 4(b) shows this approach to structure learning via first-order tree learning. The splitting criteria is the squared error over the examples and the goal is to minimize this squared error in each node as shown in 2.

$$\begin{aligned} & \min \sum_{y \in \mathbf{x}_r} \left[I(z) - E(z \notin \text{class}) - \sum_{j: x_j \in \mathbf{x}_l} \alpha_j \beta_i d_i(x_j, z) \right]^2 \\ & + \sum_{y \in \mathbf{x}_l} \left[I(z) - E(z \notin \text{class}) - \sum_{j: x_j \in \mathbf{x}_r} \alpha_j \beta_i d_i(x_j, z) \right]^2 \end{aligned} \quad (2)$$

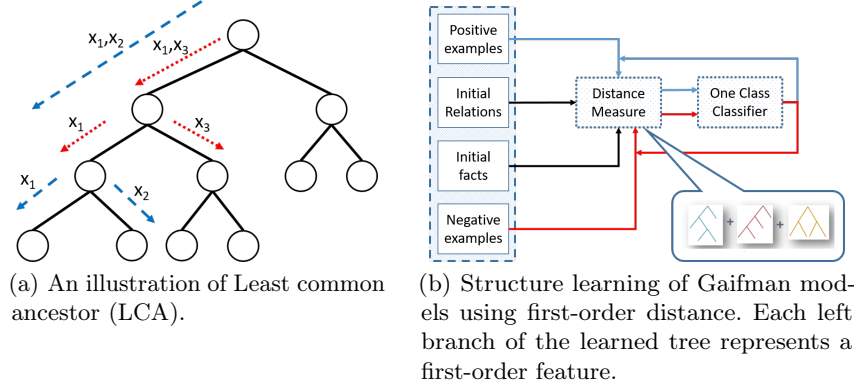


Fig. 4. Learning relational rules with FOTD.

$I(z)$ is the indicator function and returns 1 if z is an unlabeled example or 0 otherwise. Also, x_l and x_r are the examples that take the left and right branch respectively. A greedy search approach is employed for tree learning, thereby providing a *non-parametric* approach for learning these first-order trees. Algorithm 1 shows our structure learning method for DGMs using FOTD.

Procedure 1 Structure Learning using FOTD; **Input:** fact base $\mathbf{F}_s$, positive ex. $\mathbf{pos}$, negative ex. $\mathbf{neg}$

```

1: function LearnGaifmanStruct( $\mathbf{F}_s$ ,  $\mathbf{pos}$ ,  $\mathbf{neg}$ )
2:   for every  $x_1, x_2$  in  $\mathbf{pos}$  do
3:     Calculate  $d(x_1, x_2)$  according to equation 1
4:      $D(x_1, x_2) = \sum_i \beta_i d_i(x_1, x_2)$   $\triangleright$  Compute weighted distance between  $x_1, x_2$ 
5:   end for
6:   for a given new unlabeled example  $z$  do
7:      $E(z \notin \text{class}) = \sum_j \alpha_j D(x_j, z)$   $\triangleright$  Calculate the density estimate
8:   end for
9:   Learn the tree  $T$  iteratively by minimizing equation 2
10:  return LeftBranch( $T$ )
11: end function

```

3.2 Propositional Feature Construction

Once extracted, the first-order rules are instantiated (grounded) to obtain graphs G_{pos} and G_{neg} . While several *feature aggregations* exist, we employ *counts* since they have been successfully employed in many relational models. For every first-order feature $\varphi \in \Phi$, the first and last entity are instantiated corresponding to the tuples satisfying the query. For example, consider the knowledge base snippet in Fig. 1; let the positive example be `Interacts(Pravastatin, Simvastatin)`. For a relational feature, say `EnzymeInhib(d0, t0)  $\wedge$  _EnzymeInhib(t0, d1)`, and

the substitution $\{d0/Pravastatin, d1/Simvastatin\}$ we obtain the **partially-grounded feature** $\text{EnzymeInhib}(Pravastatin, t0) \wedge _EnzymeInhib(t0, Simvastatin)$.

Next, all the entities that completely satisfy this partially grounded feature are obtained. The features for each query variable are then obtained as counts of the number of entities in the satisfied grounded features that are also present in the neighborhood of the query entities in the Gaifman graph $\mathcal{G}$. For example, in the Gaifman graph in Figure 1, we check if $\text{EnzymeInhib}(Pravastatin, CytochromeP4502C9) \wedge _EnzymeInhib(CytochromeP4502C9, Simvastatin)$ satisfies the given first-order feature i.e. this grounding $\in G$. If the grounding satisfies the relational feature and since *CytochromeP4502C9* is present in the Gaifman neighborhood of *Pravastatin* (as well as *Simvastatin*), the count of the relational feature is increased by 1. Thus, for every query variable q we obtain a propositional feature $f = [f_1, \dots, f_{|\Phi|}]$ of length $|\Phi|$:

$$f_i = \begin{cases} |\psi^{N_r(q)}(q)|, & \text{if } q(e_1, e_2) \text{ partially grounds } \Phi_i, \\ 0, & \text{otherwise.} \end{cases} \quad (3)$$

Recall that ψ refers to the relativized first-order formula, and consequently $\psi^{N_r(q)}(q)$ is the r -local formula for a neighborhood N of depth r . Thus, we count the number of entities in the satisfied grounded features that are also satisfied in the neighborhood structure of the Gaifman graph.

Learning a Discriminative Model: After learning the propositional features, any standard classifier can be used for link prediction. In our experiments, we employ gradient-boosting [10] and logistic regression. The classification algorithm itself is not a key contribution of our work and as we demonstrate empirically next, a standard classifier suffices for learning an effective discriminative model.

4 Experiments

We consider 5 *real-world novel relational data sets of varying domains* (Table 1) in our empirical evaluation³ of feature generation using random walks, ILP and FOTD. We aim to answer the following questions: **Q1:** How do different feature selection strategies compare across diverse domains from different applications? **Q2:** Does choice of the discriminative algorithm impact the performance? **Q3:** How do different feature selection strategies impact performance in domains with high class imbalance? **Q4:** What are effects of locality parameters r , w and k ?

Data sets: Drug-Drug interactions (DDI): This data set [9] consists of 78 drugs obtained from DrugBank. The data set has 15 relations and the target is the *Interactions* relation between the drug entities. **Protein-Protein interactions (PPI):** This data set [20] has 7 relations and is obtained from Alchemy. The target is *interaction* relation between two protein entities. **Financial NLP:** is obtained by extracting information from *SEC Form S-1* documents, which were scraped

³ We have made our code available at <https://bit.ly/2EisPCR>

Table 1. Evaluation domains and their properties.

Data set	#Entities	#Relations	#Pos	#Neg	#RW rules	#ILP rules	#FOTD rules
DDI	355	15	2832	3188	68	36	25
PPI	797	7	1915	1915	42	5	15
Financial NLP	650	7	186	1029	222	6	25
ICML Co-Author	558	5	155	6498	7	15	7
NELL Sports	4147	6	300	600	36	15	13

and converted into relational format. This data set has 7 relations and the target the relation *sentenceContainsTarget* between sentence and word entities. **ICML Co-Author:** is obtained by mining publication data from ICML 2018; the data set has 5 relations and the target is the *CoAuthor* relation between persons. **NELL Sports:** was generated by the Never Ending Language Learner (NELL) [26] consisting of information about players and teams. It has 6 relations and the task is to predict whether a team plays a particular sport i.e *teampplayssport*.

Table 2. Results for the balanced relational domains.

Data set	Features	Accuracy		Recall		F1		AUC-ROC	
		LR	GB	LR	GB	LR	GB	LR	GB
DDI	RW	0.657	0.669	0.469	0.530	0.564	0.602	0.647	0.662
	ILP	0.696	0.774	0.467	0.674	0.592	0.729	0.684	0.767
	FOTD	0.860	0.897	0.939	0.991	0.864	0.901	0.864	0.902
PPI	RW	0.700	0.785	0.586	0.707	0.661	0.767	0.699	0.785
	ILP	0.613	0.661	0.397	0.553	0.506	0.620	0.613	0.661
	FOTD	0.727	0.733	0.996	0.999	0.785	0.789	0.727	0.733
NELL Sports	RW	0.783	0.822	0.414	0.569	0.569	0.689	0.696	0.762
	ILP	0.782	0.824	0.431	0.590	0.578	0.699	0.699	0.769
	FOTD	0.793	0.833	0.431	0.6	0.59	0.714	0.708	0.778

Results: Table 1 also shows the number of relational rules learned by different techniques. Tables 2 and 3 present the results for the balanced and unbalanced domains respectively, after 5-fold cross validation, with logistic regression (LR) and gradient boosting (GB). All experiments were run on a *64-bit Intel(R) Xeon(R) CPU E5-2630 v3* server with the parameter values $r=1$, $k=10$ and $w=5$.

To answer **Q1**, we note that FOTD outperform ILP and relational RWs across a majority of the domains. This is expected since FOTD considers the density of the positive and negative examples separately, allowing the features it generates to discriminate better. In answer to **Q2**, results in Table 2 show that choice of classifier does not make much difference in the final performance after learning relational rules, though the performance of GB is almost always higher than LR. The ICML CoAuthor (neg-to-pos ratio of 42:1) and Financial NLP (neg-to-pos ratio of 6:1) are highly imbalanced; consequently, we report AUC-PR (table 3). In both domains, AUC-PR for FOTD outperforms the other rule-construction methods by a large margin. Random walk rules, in particular, cause all the examples to be classified as negative, and results in recall and F1-score of 0 in

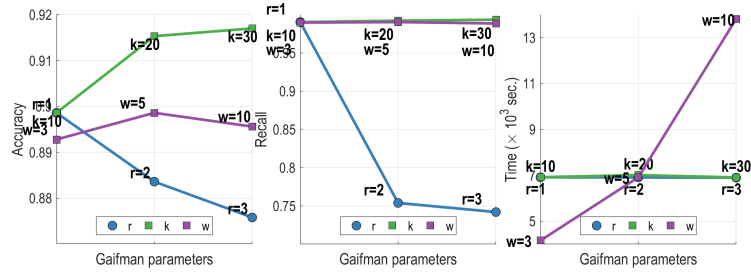


Fig. 5. (left) Accuracy, (middle) recall and (right) running time for various values of r , k and w . For varying r : $w=5$ and $k=10$, for varying w : $r=1$ and $k=10$ and for varying k : $w=5$ and $r=1$.

both the domains. Thus, to answer **Q3**: different feature selection strategies do affect the performance in highly-imbalanced domains. To answer **Q4**, figure 5

Table 3. Results for the unbalanced relational domains.

Data set	Features	Recall		F1		AUC-PR	
		LR	GB	LR	GB	LR	GB
Financial NLP	RW	0.0	0.0	0.0	0.0	0.168	0.168
	ILP	0.068	0.633	0.112	0.727	0.200	0.6023
	FOTD	0.788	0.800	0.882	0.889	0.826	0.833
ICML CoAuthor	RW	0.0	0.0	0.0	0.0	0.023	0.023
	ILP	0.272	0.339	0.427	0.506	0.289	0.356
	FOTD	0.346	0.386	0.517	0.557	0.370	0.40

show the effects of varying r (depth of neighborhoods), k (number of neighbors) and w (number of neighborhoods) on the DDI data set. Generally, k does not affect performance significantly, but increasing r causes recall report to drop sharply. This is because, with $r = 1$, entities in the query neighborhood are more tightly coupled with entities in the query variables. Another important takeaway is that the rules learned using FOTD exhibit *high clinically-relevant recall* (≈ 1) on medical data sets: DDI and PPI. This has considerable implications for bioinformatics domains as recall is the most important metric; this is because a false negative (such as a misdiagnosis) results in much more serious consequences [9] than a false positive. Finally, from figure 5 (right), we note that varying r and k does not change the run time. However, increasing w increases the run time since the size of the neighborhood graph to be searched increases.

5 Conclusion

We considered the problem of full model learning of discriminative Gaifman models. Specifically, we considered three types of rule learning methods for structure learning of Gaifman models from relational data. Our algorithm first constructs a set of first-order rules, identifies the appropriate instantiations and finally counts the number of groundings per rule. These become the raw features based on which one could train a discriminative classifier. Our work provides a

method of constructing relational embeddings in an effective manner that can be combined with a scalable local model. One possible feature direction could be to consider more base model learners. Another direction is employing more graph based embedding methods that can integrate with Gaifman’s locality principle. Finally, evaluating on more real databases and knowledge graphs is an interesting direction for future research.

References

1. M. Al Hasan and M. J. Zaki. A survey of link prediction in social networks. In *Social network data analytics*. 2011.
2. H. Blockeel and L. De Raedt. Top-down induction of first-order logical decision trees. *Artificial intelligence*, 1998.
3. H. Blockeel and L. D. Raedt. Top-down induction of first-order logical decision trees. *Artificial Intelligence*, 1998.
4. M. Bröcheler, L. Mihalkova, and L. Getoor. Probabilistic similarity logic. In *UAI*, 2010.
5. H. Cai, V. W. Zheng, and K. C. Chang. A comprehensive survey of graph embedding: Problems, techniques, and applications. *IEEE Transactions on Knowledge and Data Engineering*, 2018.
6. P. M. Chuan, M. Ali, et al. Link prediction in co-authorship networks based on hybrid content similarity metric. *Applied Intelligence*, 2018.
7. C. P. De Campos, Z. Zeng, and Q. Ji. Structure learning of bayesian networks using constraints. In *ICML*, 2009.
8. L. De Raedt, A. Kimmig, and H. Toivonen. Problog: A probabilistic prolog and its application in link discovery. 2007.
9. D. S. Dhami, G. Kunapuli, M. Das, D. Page, and S. Natarajan. Drug-drug interaction discovery: Kernel learning from heterogeneous similarities. *Smart Health*, 2018.
10. J. H. Friedman. Greedy function approximation: a gradient boosting machine. *Annals of statistics*, 2001.
11. H. Gaifman. On local and non-local properties. In *Studies in Logic and the Foundations of Mathematics*. 1982.
12. L. Getoor and B. Taskar. *Introduction to statistical relational learning*. MIT press Cambridge, 2007.
13. P. Goyal and E. Ferrara. Graph embedding techniques, applications, and performance: A survey. *Knowledge-Based Systems*, 2018.
14. S. Harada, H. Akita, et al. Dual convolutional neural network for graph of graphs link prediction. *arXiv preprint arXiv:1810.02080*, 2018.
15. T. Jaakkola, D. Sontag, A. Globerson, and M. Meila. Learning bayesian network structure using lp relaxations. In *AISTATS*, 2010.
16. N. Kaur, G. Kunapuli, T. Khot, K. Kersting, W. Cohen, and S. Natarajan. Relational restricted boltzmann machines: A probabilistic logic learning approach. In *ILP*, 2017.
17. T. Khot, S. Natarajan, K. Kersting, and J. Shavlik. Learning markov logic networks via functional gradient boosting. In *2011 IEEE 11th International Conference on Data Mining*, 2011.
18. T. Khot, S. Natarajan, and J. W. Shavlik. Relational one-class classification: A non-parametric approach. In *AAAI*, 2014.

19. S. Kok and P. Domingos. Learning markov logic network structure via hypergraph lifting. In *ICML*, 2009.
20. S. Kok, M. Sumner, M. Richardson, P. Singla, H. Poon, D. Lowd, J. Wang, and P. Domingos. The alchemy system for statistical relational {AI}. 2009.
21. N. Lao and W. W. Cohen. Relational retrieval using a combination of path-constrained random walks. *Machine learning*, 2010.
22. N. Lao, T. Mitchell, and W. W. Cohen. Random walk inference and learning in a large scale knowledge base. In *EMNLP*, 2011.
23. P. Larrañaga et al. Structure learning of bayesian networks by genetic algorithms: A performance analysis of control parameters. *TPAMI*, 1996.
24. S.-I. Lee, V. Ganapathi, and D. Koller. Efficient structure learning of markov networks using l_1 -regularization. In *NIPS*, 2007.
25. V. Martínez, F. Berzal, and J.-C. Cubero. A survey of link prediction in complex networks. *ACM Computing Surveys (CSUR)*, 2017.
26. T. Mitchell, W. Cohen, et al. Never-ending learning. *Communications of the ACM*, 2018.
27. S. Muggleton. Inductive logic programming. *New generation computing*, 1991.
28. S. Muggleton. Inverse entailment and prolog. *New generation computing*, 1995.
29. S. Natarajan, T. Khot, K. Kersting, B. Gutmann, and J. Shavlik. Gradient-based boosting for statistical relational learning: The relational dependency network case. *Machine Learning*, 2012.
30. S. Natarajan, G. Kunapuli, C. O'Reilly, R. Maclin, T. Walker, D. Page, and J. Shavlik. Ilp for bootstrapped learning: A layered approach to automating the ilp setup problem. In *ILP*, 2009.
31. M. Nickel, K. Murphy, V. Tresp, and E. Gabrilovich. A review of relational machine learning for knowledge graphs. *Proceedings of the IEEE*, 2016.
32. M. Niepert. Discriminative gaussian models. In *NIPS*, 2016.
33. J. R. Quinlan. Learning logical definitions from relations. *Machine learning*, 1990.
34. L. D. Raedt, K. Kersting, S. Natarajan, and D. Poole. Statistical relational artificial intelligence: Logic, probability, and computation. *Synthesis Lectures on Artificial Intelligence and Machine Learning*, 2016.
35. M. Richardson and P. Domingos. Markov logic networks. *Machine learning*, 2006.
36. A. Srinivasan. The aleph manual, 2001.
37. B. Taskar, M.-F. Wong, P. Abbeel, and D. Koller. Link prediction in relational data. In *NIPS*, 2004.
38. Q. Wang, Z. Mao, B. Wang, and L. Guo. Knowledge graph embedding: A survey of approaches and applications. *IEEE Transactions on Knowledge and Data Engineering*, 2017.
39. Y. Zhou. Structure learning of probabilistic graphical models: a comprehensive survey. *arXiv preprint arXiv:1111.6925*, 2011.